МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ

по лабораторной работе №1
по дисциплине «Построение и анализ алгоритмов»
Тема: Поиск с возвратом. Вариант 2р.

Студент гр. 4345 Лунченко Н.С.
Преподаватель Жангиров Т.Р.

Санкт-Петербург
2026

Цель работы:

Изучить рекурсивный бэктрекинг. Исследование времени выполнения от размера квадрата

Постановка задачи

У Вовы много квадратных обрезков доски. Их стороны (размер) изменяются от

1 до N −1, и у него есть неограниченное число обрезков любого размера. Но ему очень хочется получить большую столешницу - квадрат размера N. Он может получить ее, собрав из уже имеющихся обрезков(квадратов).

Например, столешница размера

7×7 может быть построена из 9 обрезков.

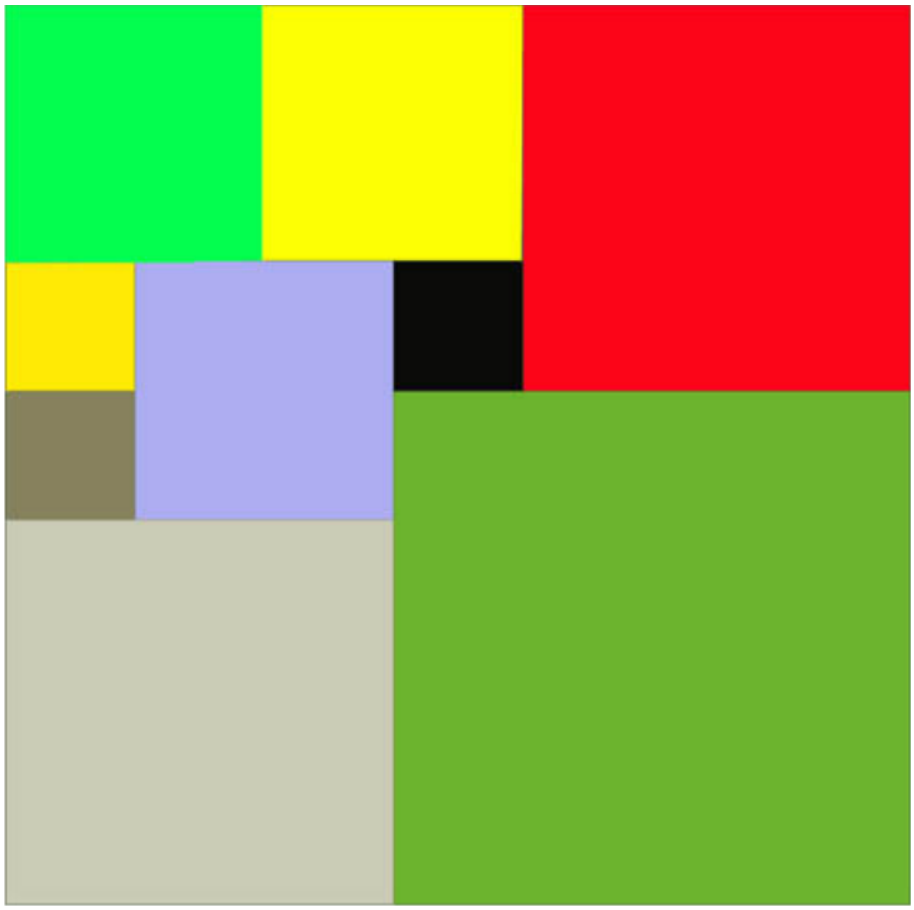

Рис. 1 – Пример решения

Внутри столешницы не должно быть пустот, обрезки не должны выходить за пределы столешницы и не должны перекрываться. Кроме того, Вова хочет использовать минимально возможное число обрезков.

Входные данные

Размер столешницы - одно целое число

N (2≤N≤20).

Выходные данные

Одно число K, задающее минимальное количество обрезков(квадратов), из которых можно построить столешницу(квадрат) заданного размера N. Далее должны идти K строк, каждая из которых должна содержать три целых числа x, y и w, задающие координаты левого верхнего угла ($1 \le x,y \le N$) и длину стороны соответствующего обрезка(квадрата).

Пример входных данных

7

Соответствующие выходные данные

9

1 1 2

1 3 2

3 1 1

4 1 1

3 2 2

5 1 3

4 4 4

1 5 3

3 4 1

Выполнение работы

Базовое представление данных:

Поле представляется как массив целых чисел bin_field длины N, где каждый бит числа соответствует ячейке 1×1, где 1 - ячейка занята, 0 - ячейка свободна. Такое представление позволяет проверить возможность размещения за O(1) для строки (O(N) для поля).

Реализованные структуры данных:

Класс Square: представление размещенного квадрата

x, y - координаты левого верхнего угла

side - длина стороны

Класс Position: координаты свободной ячейки

Класс Field: состояние поля разбиения

bin_field - битовое представление занятых ячеек (int массив)

line_mask - маска для выделения битов в строке

current_solution и best_solution - текущее и лучшее решения

count и best_count - текущее и минимальное количество квадратов

Основные алгоритмы

1. Проверка возможности размещения (can_place)

Использует битовые маски для быстрой проверки пересечений

Временная сложность: O(side)

2. Размещение/удаление квадрата (place, remove)

Побитовые операции для модификации поля

Временная сложность: O(side)

3. Поиск первого свободного места (get_first_free_position)

Поиск первого нулевого бита в строке

Временная сложность: $O(side^2)$ в худшем случае

4. Определение максимального размера квадрата (get_max_square_at_position)

Поиск наибольшего возможного квадрата в позиции

Временная сложность: $O(side^2)$

Пошаговое описание работы

1. Предварительная обработка размер. Если N составное и имеет простой делитель p, задача сводится к решению для размера p с последующим масштабированием в q = N/p раз.
2. Предзаполнение в зависимости от свойств:

   Для простых N:

   Помещаем квадрат (⌊N/2⌋+1) в левый верхний угол

   2. Размещаем два квадрата ⌊N/2⌋ справа и снизу

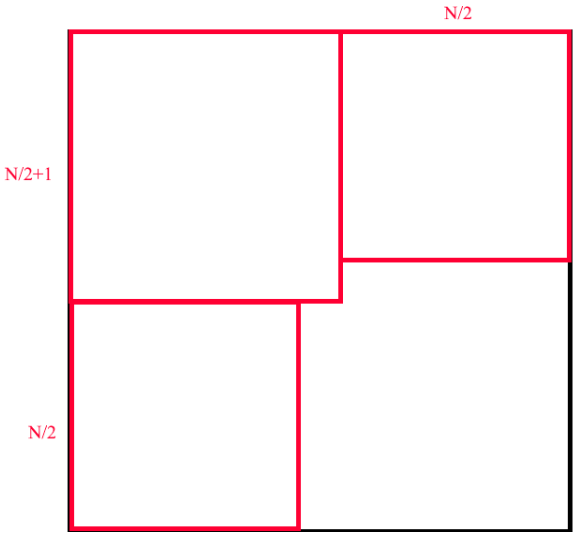

Рис 2. – Предзаполнение для простых N

   3. Если ⌊N/2⌋ - нечетное, добавляем квадраты 1×1 и 2×2

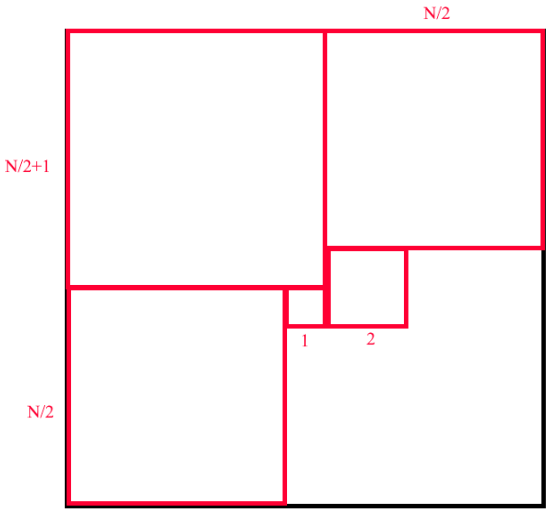

Рис 3 – Предзаполнение для простых N при ⌊N/2⌋ - нечетное

Для четных N:

Разбиваем на 4 равных квадрата N/2 × N/2

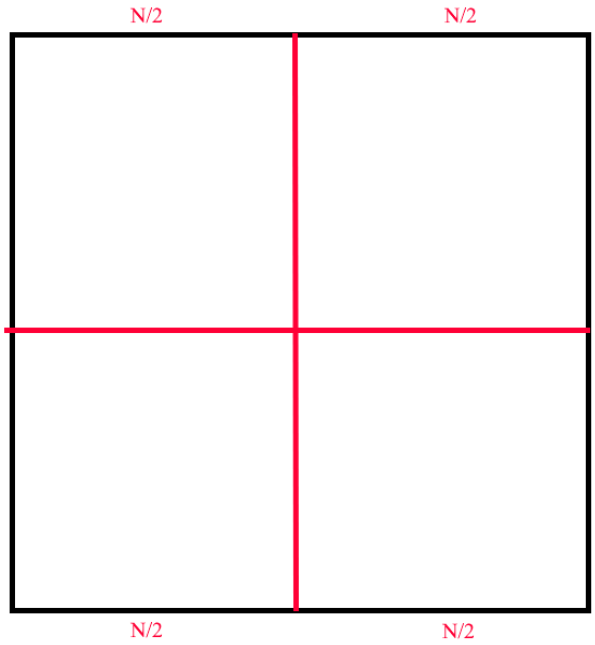

Рис. 4 – Предзаполнение для четных N

Для N, кратных 3:

Пусть N = a·b, где a > b

- Размещаем прямоугольный блок a×(b-1) вверху
- Заполняем правую колонку b квадратами a×a

- Заполняем нижнюю строку (b-1) квадратами a×a

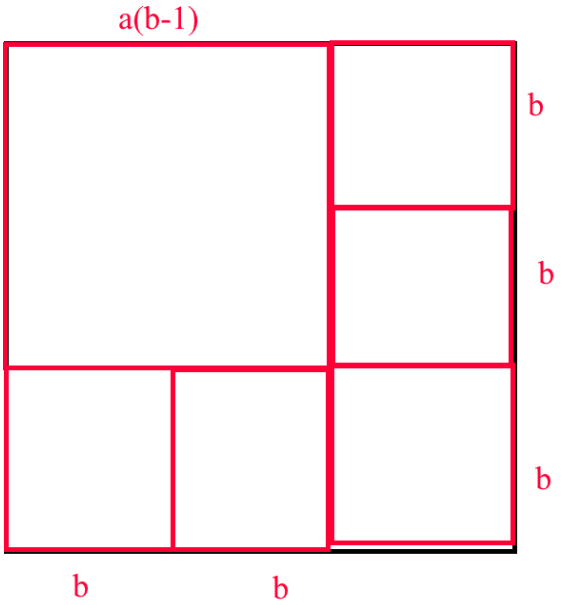

Рис. 5 – Предзаполнение для кратных 3

3. Рекурсивный поиск проходит в функции с сигнатурой:

def find_minimal_parts_recursive(field: Field) -> None

Функция осуществляет перебор квадратов от максимального к минимальному

с Backtracking и отсечением по текущему лучшему решению

Алгоритм перебора:

1. Отсечение по текущему лучшему решению
2. Проверка заполненности поля
3. Поиск первой свободной позиции
4. Определение максимально возможного квадрата
5. Перебор от большего к меньшем

   5.1 Размещение квадрата

   5.2 Рекурсивный вызов

   5.3 Откат

Для оптимизации алгоритма были использованы:

1. Битовое представление строки поля, что ускоряет проверку пересечений, так как не требует поэлементного перебора ячеек, и поиск первого свободного места. Эти действия выполняется за O(1).

2. Предварительное заполнение по шаблонам, что позволяет провести алгоритм для четных и чисел делящихся для 3 за O(1), а для простых чисел ускорить в 4 раза.
3. Отсечения ветви, глубина которой больше или равна лучшему результату, что ограничивает глубину рекурсии
4. Перебор от больший к меньшим квадратам, что позволяет быстрее отсечь ветви.

Оценка сложности алгоритма

Для N кратных 2 и 3 сложность линейная, так как заполняются одинаковым количеством квадратов, но чем больше N – тем для большого количества строк нужно произвести заполение => O(N)

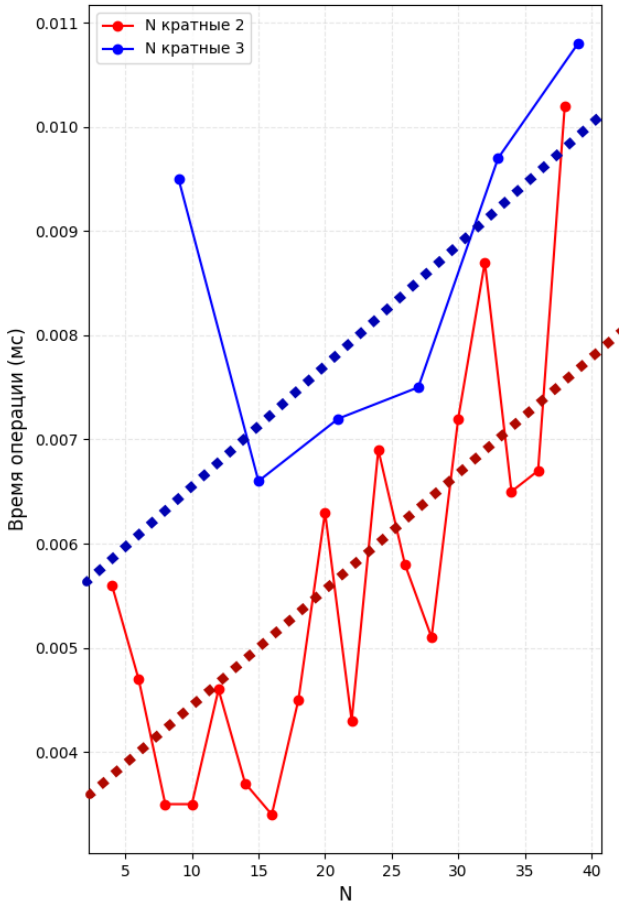

Рис 6. – График T(N) для N кратных 2 и 3

Для простых N сложность экспоненциальная, так как в худшем случае для каждой ветки рекурсии мы в каждой свободной клетке мы рассматриваем N/4 квадратов $O(c^{N/4})$. Для составных N ситуация

схожая, находим простой делитель находим решение для него и масштабируем.

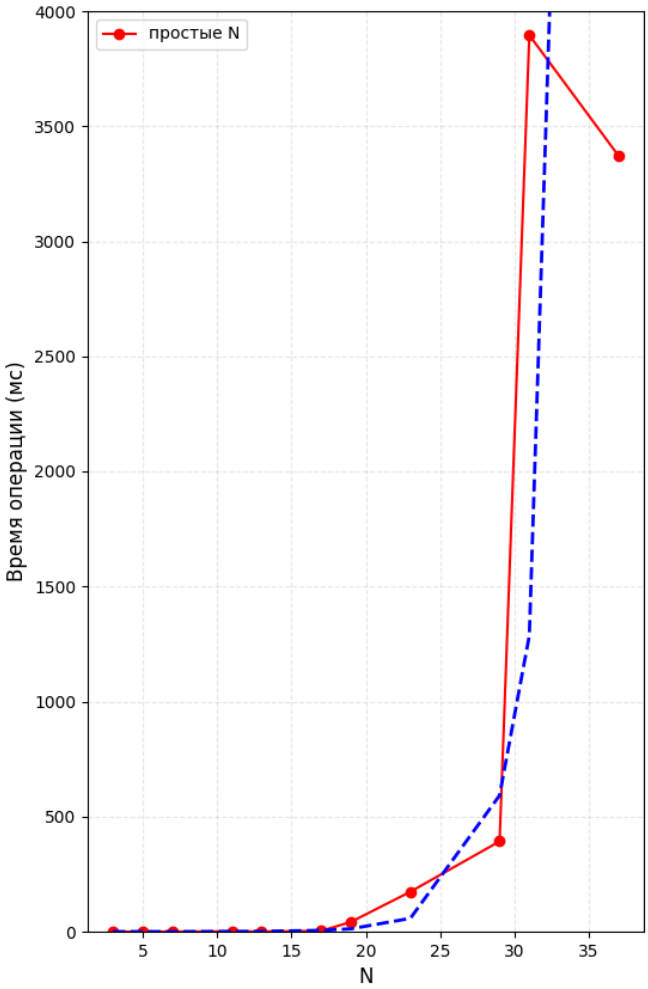

Рис 7. – График T(N) для простых N

В каждом случае сложность в памяти равна:

В худшем случае $O(N^2)$ – если заполнен только квадратами 1x1 – кол-во квадратов N^2.

В среднем случае O(N), так количество квадратов пропорционально N.

Исходный код программы см. в приложении А.

Результаты тестирования см. в Таблице 1.

Таблица 1 – Результаты тестирования

№	Входные данные	Выходные данные	Комментарий
1	3	6 1 1 2	Простое число, но так как само является 3, то

		1 3 1 3 1 1 3 2 1 2 3 1 3 3 1	применим алгоритм предзаполнения для чисел кратных 3-ем
2	38	4 1 1 19 20 1 19 1 20 19 20 20 19	Четное – всегда разделяется на 4 квадрата
3	5	8 1 1 3 1 4 2 4 1 2 3 4 1 4 3 2 3 5 1 4 5 1 5 5 1	Простое
4	25	8 1 1 15 1 16 10 16 1 10 11 16 5 16 11 10 11 21 5 16 21 5 21 21 5	Составное, нашли простой делитель 5, и применили масштабирование

Выводы

Разработанная программа реализует поиск с возвратом для разбиения квадратной доски на минимальное число меньших квадратов. Алгоритм перебирает все возможные варианты размещения, начиная с наиболее крупных квадратов для сужения области поиска.

Несмотря на экспоненциальную сложность в худшем случае (из-за быстрого роста числа комбинаций), выбор больших квадратов и отсечения заведомо неперспективных ветвей позволяет снизить реальную сложность до O(c^(N/4)). Это делает метод эффективным для досок среднего размера, гарантируя нахождение оптимального решения.

ПРИЛОЖЕНИЕ А
ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.py

```
import matplotlib.pyplot as plt
import time
import numpy as np


class Square:
    def __init__(self, x=0, y=0, side=0):
        self.x = x
        self.y = y
        self.side = side

class Position:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

class Field:
    def __init__(self, side):
        self.count = 0
        self.best_count = float('inf')
        self.side = side
        self.bin_field = [0] * side
        self.line_mask = (1 << side) - 1
        self.squares = []
        self.best_squares = []
        self.current_solution = []
        self.best_solution = []

def can_place(field, square_side, pos):
    if not (pos.x + square_side <= field.side  and pos.y + square_side
<= field.side):
        return False
    line_mask = ((1 << square_side) - 1) << pos.x
```

```
    for i in range(square_side):
        if field.bin_field[pos.y + i] & line_mask:
            return False
    return True

def place(field, line_mask, square_side, y_coord):
    for i in range(square_side):
        field.bin_field[y_coord + i] |= line_mask

def remove(field, line_mask, square_side, y_coord):
    for i in range(square_side):
        field.bin_field[y_coord + i] &= ~line_mask

def get_first_free_position(field):
    for y in range(field.side):
        free = ~field.bin_field[y] & field.line_mask
        if free:
            x = (free & -free).bit_length() - 1
            return Position(x, y)
    return Position(0, 0)

def is_grid_full(field):
    for i in range(field.side):
        if (field.bin_field[i] & field.line_mask) != field.line_mask:
            return False
    return True

def get_max_square_at_position(field, pos):
    dx = field.side - pos.x
    dy = field.side - pos.y
    max_possible = min(dx, dy)
    if max_possible > field.side - 1:
        max_possible = field.side - 1

    for side in range(max_possible, 0, -1):
        if can_place(field, side, pos):
            return side
    return 0
```

```
def find_minimal_parts_recursive(field):
    if field.count >= field.best_count:
        return

    if is_grid_full(field):
        field.best_solution = field.current_solution.copy()
        field.best_count = field.count
        print(f"Лучший способ - {field.best_count} квадратов")
        return
    pos = get_first_free_position(field)
    max_side = get_max_square_at_position(field, pos)
    for side in range(max_side, 0, -1):
        line_mask = ((1 << side) - 1) << pos.x

        place(field, line_mask, side, pos.y)
        field.current_solution.append(Square(pos.x  +  1,  pos.y  +  1,
side))
        field.count += 1

        find_minimal_parts_recursive(field)

        remove(field, line_mask, side, pos.y)
        field.current_solution.pop()
        field.count -= 1

def is_prime(N):
    if N < 2:
        return 0
    for i in range(2,int(N**0.5)+1):
        if N%i==0:
            return 0
    return 1

def find_max_b(N):
    for i in range(N//2, 2, -1):
        if N%i == 0:
            return N//i
```

```
def prefill_primary(field,N):
    a = N // 2 + 1
    b = N // 2
    place(field, ((1 << a) - 1) << 0, a, 0)
    field.current_solution.append(Square(1, 1, a))
    field.count += 1

    place(field, ((1 << b) - 1) << 0, b, a)
    field.current_solution.append(Square(1, a + 1, b))
    field.count += 1

    place(field, ((1 << b) - 1) << a, b, 0)
    field.current_solution.append(Square(a + 1, 1, b))
    field.count += 1

    if (a%2):
        place(field, ((1 << 1) - 1) << b, 1, a)
        field.current_solution.append(Square(b + 1, a+1,1))
        field.count += 1

        place(field, ((1 <<2) - 1) << a, b, 2)
        field.current_solution.append(Square(a+1, b + 1,2))
        field.count += 1
    print("Выполнено предзаполнение для простых чисел")

def prefill_even(field,N):
    place(field, ((1 << N//2) - 1) << 0, N//2, 0)
    field.current_solution.append(Square(1 , 1, N//2))
    field.count += 1

    place(field, ((1 << N//2) - 1) << N//2, N//2, 0)
    field.current_solution.append(Square(1+N//2 , 1, N//2))
    field.count += 1

    place(field, ((1 << N//2) - 1) << 0, N//2, N//2)
    field.current_solution.append(Square(1 , 1+N//2, N//2))
```

```
    field.count += 1

    place(field, ((1 << N // 2) - 1) << N//2, N//2, N // 2)
    field.current_solution.append(Square(1 + N//2, 1 + N // 2, N // 2))
    field.count += 1
    print("Выполнено предзаполнение для числе кратных 2")


def prefill_three(field,N):
    b = find_max_b(N)
    a = N//b

    place(field, ((1 <<  a*(b-1)) - 1) << 0, a*(b-1), 0)
    field.current_solution.append(Square(1, 1, a*(b-1)))
    field.count += 1

    for i in range (0,b):
        place(field, ((1 << a) - 1) << N-a, a, i*a)
        field.current_solution.append(Square(N-a+1, 1+i*a, a))
        field.count += 1
    for i in range (0,b-1):
        place(field, ((1 << a) - 1) << i*a, a, N-a)
        field.current_solution.append(Square(1+i*a, N-a+1, a))
        field.count += 1
    print("Выполнено предзаполнение для числе кратных 3")

def prefill_known_patterns(field):
    N = field.side
    if is_prime(N): prefill_primary(field, N)
    elif N%2==0: prefill_even(field, N)
    elif N%3==0: prefill_three(field, N)


def reduce_to_prime_base(N):
    for p in range(3, int(N**0.5) + 1):
        if N % p == 0 and is_prime(p):
            return p, N // p
```

```
    return N, 1

def print_solution(field, scale):
    print(field.best_count)

    for sq in field.best_solution:
        x = sq.x
        y = sq.y
        s = sq.side

        if scale > 1:
            if x != 1:
                x = (x - 1) * scale + 1
            if y != 1:
                y = (y - 1) * scale + 1
            s *= scale

        print(f"{x} {y} {s}")


def find_minimum_partition():
    try:
        N = int(input())
        if N < 2 or N > 63:
            return 1
    except:
        return 1

    field = Field(N)
    scale = 1
    if (field.count == 0 and N%2):
        p, q = reduce_to_prime_base(N)

        if q > 1:
            scale = q
            N = p
            field = Field(N)
    print(f"Масштабирование - {scale}\nРассматриваем N = {N}")
```

```
    prefill_known_patterns(field)

    if not is_grid_full(field):
        find_minimal_parts_recursive(field)
    else:
        print("Поле полностью заполнено")
        field.best_solution = field.current_solution.copy()
        field.best_count = field.count
    print_solution(field, scale)


def create_graphics():
    Times = []
    Times_complex = []
    Times_primary = []
    Primary_values = []
    N_values = list(range(3, 40))
    Complex_values = []

    for N in N_values:
        field = Field(N)
        scale = 1
        start = time.perf_counter() * 1e3
        if (field.count == 0 and N % 2 and N%3):
            p, q = reduce_to_prime_base(N)

            if q > 1:
                scale = q
                N = p
                field = Field(N)
        prefill_known_patterns(field)

        if not is_grid_full(field):
            find_minimal_parts_recursive(field)
        else:
            field.best_solution = field.current_solution.copy()
            field.best_count = field.count
        end = time.perf_counter() * 1e3
```

```
        Times.append(end - start)
        if is_prime(N*scale):
            Primary_values.append(N*scale)
            Times_primary.append(end - start)
        elif scale != 1:
            Complex_values.append(N*scale)
            Times_complex.append(end - start)

    plt.figure(figsize=(6, 10))
    plt.plot(N_values,  Times,  'ro-',  markersize=6,  linewidth=1.5,
label='Все N')
    plt.grid(True, alpha=0.3, linestyle='--')
    plt.xlabel('N', fontsize=12)
    plt.ylabel('Время операции (мс)', fontsize=12)
    plt.legend()
    plt.savefig('Plot.png')
    plt.show()

    plt.figure(figsize=(6, 10))
    plt.plot([N_values[i] for i in range(1, len(Times), 2)], [Times[i]
for i in range(1,len(N_values), 2)], 'ro-', markersize=6, linewidth=1.5,
label='N кратные 2')
    plt.plot([N_values[i]  for  i  in  range(6,  len(N_values),  6)],
[Times[i] for i in range(6, len(N_values), 6)], 'bo-', markersize=6,
linewidth=1.5, label='N кратные 3')
    plt.grid(True, alpha=0.3, linestyle='--')
    plt.xlabel('N', fontsize=12)
    plt.ylabel('Время операции (мс)', fontsize=12)
    plt.legend()
    plt.savefig('Plot_full.png')
    plt.show()

    plt.figure(figsize=(6, 10))
    plt.plot(Complex_values,   Times_complex,   'ro-',   markersize=6,
linewidth=1.5, label='Составные N')
    plt.grid(True, alpha=0.3, linestyle='--')
    plt.xlabel('N', fontsize=12)
    plt.ylabel('Время операции (мс)', fontsize=12)
```

```
    plt.legend()
    plt.savefig('Plot_complex.png')
    plt.show()

    plt.figure(figsize=(6, 10))
    plt.plot(Primary_values,   Times_primary,   'ro-',   markersize=6,
linewidth=1.5, label='простые N')
    plt.grid(True, alpha=0.3, linestyle='--')
    plt.xlabel('N', fontsize=12)
    plt.ylabel('Время операции (мс)', fontsize=12)
    plt.legend()
    z = np.polyfit(Primary_values, np.log(Times_primary), 1)
    trend = np.exp(z[1]) * np.exp(z[0] * np.array(Primary_values))
    plt.plot(Primary_values, trend, 'b--', linewidth=2,
             label='Экспонента')
    plt.ylim(0, 4000)
    plt.savefig('Plot_primary.png')
    plt.show()
    print(Times)

if __name__ == "__main__":
    print("1 - Алгоритм для задаваемого N\n2 - Построение графиков")
    if (input() == "1"):
        find_minimum_partition()
    else:
        create_graphics()
```